

Neural Network Fundamentals

From Perceptron to Gradient Descent

Perceptron থেকে Gradient Descent পর্যন্ত — একটি সুশৃঙ্খল ধারাবাহিক লার্নিং পথ

সূচিপত্র (Table of Contents)

Contents

Chapter 1 · Perceptron ও Neuron	4
১.১ Perceptron	4
১.২ Linear Combination	4
১.৩ Decision Boundary	5
১.৪ Perceptron-এর সীমাবদ্ধতা	5
১.৫ Neuron	6
১.৬ Perceptron বনাম Neuron	6
Chapter 2 · Activation Functions	8
২.১ Activation Function	8
২.২ Step Function	8
২.৩ Sigmoid	8
২.৪ Tanh	9
২.৫ ReLU	9
২.৬ Leaky ReLU	9
২.৭ Softmax	10
২.৮ কেন Non-linearity প্রয়োজন?	10
Chapter 3 · Neural Network Architecture	12
৩.১ Input Layer	12
৩.২ Feature	12
৩.৩ Image Input	12
৩.৪ Flatten	13
৩.৫ Batch Shape	13
৩.৬ Hidden Layer	13
৩.৭ Fully Connected / Dense Layer	14
৩.৮ একাধিক Neuron	14
৩.৯ ম্যাট্রিক্স আকারে প্রকাশ (Matrix Form)	14
Chapter 4 · Parameters	16
৪.১ Weight	16
৪.২ Bias	16
৪.৩ Parameter গণনা	16

৪.৪ বৃহৎ Parameter-বিশিষ্ট Model	17
Chapter 5 · Output Layer	18
৫.১ Regression → Linear	18
৫.২ Binary Classification → Sigmoid	18
৫.৩ Multi-class Classification → Softmax	18
৫.৪ Multi-label Classification → Sigmoid	18
৫.৫ Probability	18
৫.৬ Threshold	19
সারাংশ সারগি	19
Chapter 6 · Loss Functions	20
৬.১ Loss কী?	20
৬.২ কেন Loss প্রয়োজন?	20
৬.৩ MSE (Mean Squared Error)	20
৬.৪ MAE (Mean Absolute Error)	20
৬.৫ Huber Loss	21
৬.৬ Binary Cross-Entropy	21
৬.৭ Cross-Entropy (Multi-class)	21
৬.৮ $-\log(x)$ -এর আচরণ	22
Chapter 7 · Gradient Descent	23
৭.১ Gradient	23
৭.২ Partial Derivative	23
৭.৩ Weight Update	23
৭.৪ Bias Update	24
৭.৫ Gradient Descent	24
সম্পূর্ণ প্রক্রিয়ার সংযোগ	24
সম্পূর্ণ পাইপলাইনের ব্যাখ্যা	25

Chapter 1 · Perceptron ও Neuron

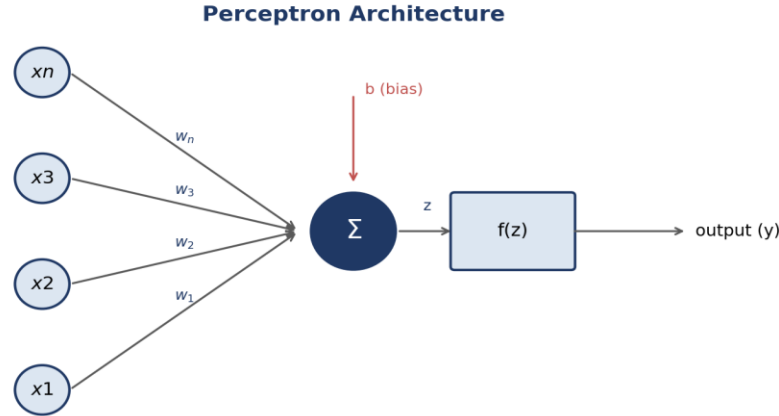
এই বইয়ের সমগ্র শিক্ষণ-প্রক্রিয়া শুরু হচ্ছে সবচেয়ে মৌলিক একক — Perceptron — থেকে। Neural Network-এর ভেতরে যা কিছু ঘটে, তার সবকিছুর ভিত্তি এই একটি সরল কাঠামোর মধ্যেই লুকিয়ে আছে। এই অধ্যায়ে আমরা Perceptron-এর গঠন, তার সীমাবদ্ধতা এবং সেই সীমাবদ্ধতা থেকে কীভাবে Neuron ধারণাটির উদ্ভব হলো, তা ধাপে ধাপে দেখাবো।

১.১ Perceptron

Perceptron হলো একটি Neural Network-এর সবচেয়ে ছোট গণনা-একক (computational unit), যা একাধিক Input নিয়ে একটি একক Output তৈরি করে। এটি মূলত একটি সিদ্ধান্ত গ্রহণকারী (decision-making) একক, যা Input-গুলোর ভিত্তিতে দুইটি সম্ভাব্য শ্রেণির (class) মধ্যে একটি বেছে নেয়।

একটি Perceptron-এর মৌলিক কাঠামোতে নিচের অংশগুলো থাকে:

- **Inputs ($x_1, x_2, \dots, x_n$):** যে সংখ্যাসূচক তথ্য Perceptron গ্রহণ করে।
- **Weights ($w_1, w_2, \dots, w_n$):** প্রতিটি Input-এর গুরুত্ব নির্ধারণ করে এমন একটি সংখ্যা।
- **Bias (b):** Decision Boundary-কে প্রয়োজন অনুযায়ী সরানোর জন্য ব্যবহৃত একটি অতিরিক্ত প্যারামিটার।
- **Summation:** সব Input ও তাদের Weight-এর গুণফল যোগ করে একটি একক মান তৈরি করা হয়।
- **Output:** যোগফলের ভিত্তিতে Perceptron একটি চূড়ান্ত সিদ্ধান্ত (০ অথবা ১) প্রদান করে।



চিত্র ১.১ — Perceptron-এর মৌলিক কাঠামো: Input, Weight, Bias, Summation এবং Output।

১.২ Linear Combination

Perceptron-এর ভেতরে প্রথম যে গাণিতিক কাজটি ঘটে, তাকে বলা হয় Linear Combination। এখানে প্রতিটি Input-কে তার নিজস্ব Weight দিয়ে গুণ করে, সবগুলো গুণফল যোগ করা হয়, এবং শেষে একটি Bias যোগ করা হয়।

$$z = x_1w_1 + x_2w_2 + \dots + x_nw_n + b$$

এখানে —

z হলো Linear Combination-এর ফলাফল (এখনো কোনো Activation প্রয়োগ করা হয়নি)।

$x_1, x_2, \dots, x_n$ হলো প্রতিটি Input Feature-এর মান।

$w_1, w_2, \dots, w_n$ হলো সংশ্লিষ্ট Weight, যা নির্দেশ করে সংশ্লিষ্ট Input কতটা গুরুত্বপূর্ণ।

b হলো Bias, যা পুরো সমীকরণটিকে একটি নির্দিষ্ট মাত্রায় ঊর্ধ্বে বা নিম্নে স্থানান্তরিত করে।

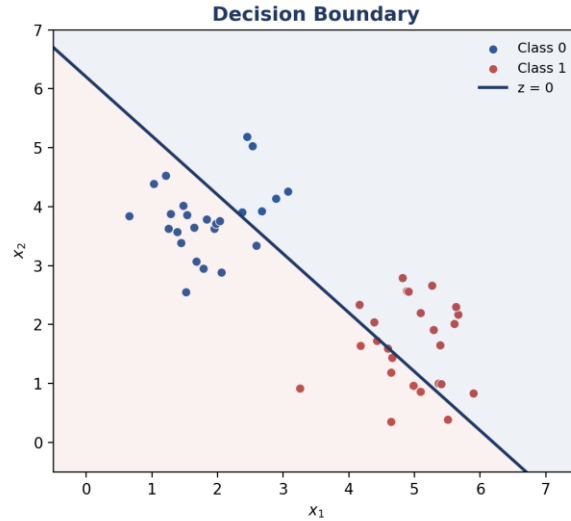
১.৩ Decision Boundary

Decision Boundary হলো সেই সীমারেখা বা তল, যা একটি Feature Space-কে দুইটি অঞ্চলে ভাগ করে — একটি অঞ্চলের প্রতিটি বিন্দু একটি Class-এর অন্তর্গত, অন্য অঞ্চলের বিন্দুগুলো অন্য Class-এর অন্তর্গত।

Perceptron-এর ক্ষেত্রে এই সীমারেখাটি ঠিক সেখানে তৈরি হয়, যেখানে $z = 0$ । কারণ z -এর চিহ্ন (positive বা negative) অনুযায়ী Perceptron সিদ্ধান্ত নেয় কোন Class নির্বাচন করা হবে।

$$z = 0 \Rightarrow x_1w_1 + x_2w_2 + b = 0$$

দুইটি Input-বিশিষ্ট (2D) ক্ষেত্রে এই সমীকরণটি একটি সরলরেখা (line) প্রতিনিধিত্ব করে। তিন বা ততোধিক Input-এর ক্ষেত্রে এটি একটি প্লেন (plane) বা হাইপারপ্লেন (hyperplane)-এ পরিণত হয়। নিচের চিত্রে ২-মাত্রিক উদাহরণে এই ধারণাটি দেখানো হলো।



চিত্র ১.২ — দুইটি Class-কে পৃথককারী Decision Boundary ($z = 0$ রেখা)।

১.৪ Perceptron-এর সীমাবদ্ধতা

Perceptron শুধুমাত্র একটি সরলরৈখিক (linear) Decision Boundary তৈরি করতে পারে। ফলে যেসব সমস্যার সমাধানে একটি সরলরেখা বা হাইপারপ্লেন দিয়ে দুই Class-কে আলাদা করা সম্ভব নয়, সেসব সমস্যা Perceptron সমাধান করতে পারে না।

এই সীমাবদ্ধতা বোঝার সবচেয়ে ক্লাসিক উদাহরণ হলো XOR সমস্যা। XOR ফাংশনের সত্যক সারণি (truth table) নিচে দেখানো হলো:

x_1	x_2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0

এই চারটি বিন্দু এমনভাবে সাজানো, যে কোনো একটিমাত্র সরলরেখা দিয়ে (0,1) ও (1,0)-কে একদিকে, এবং (0,0) ও (1,1)-কে অন্যদিকে সম্পূর্ণভাবে আলাদা করা যায় না। যেহেতু Perceptron কেবল একটি সরলরৈখিক Decision Boundary তৈরি করতে সক্ষম, তাই এটি XOR সমস্যার সমাধান করতে ব্যর্থ হয়। এই সীমাবদ্ধতাই পরবর্তীতে আরও সাধারণ (generalized) এবং শক্তিশালী একক — Neuron — এবং Non-linear Activation Function-এর প্রয়োজনীয়তা তৈরি করে।

১.৫ Neuron

Neuron হলো Perceptron-এর একটি সাধারণীকৃত (generalized) রূপ। কাঠামোগতভাবে Neuron এবং Perceptron প্রায় একই — উভয় ক্ষেত্রেই Weighted Sum এবং Bias গণনা করা হয়। মূল পার্থক্য হলো, Perceptron শুধুমাত্র Step Function ব্যবহার করে সিদ্ধান্ত নেয়, কিন্তু Neuron যেকোনো ধরনের Activation Function ব্যবহার করতে পারে।

একটি Neuron-এর সাধারণ গঠন:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b, \quad a = f(z)$$

এখানে f হলো একটি Activation Function (যেমন Sigmoid, Tanh, ReLU ইত্যাদি), এবং a হলো Neuron-এর চূড়ান্ত Output। এই a -কে Activation বলা হয়।

১.৬ Perceptron বনাম Neuron

বিষয়	Perceptron	Neuron
Activation Function	শুধুমাত্র Step Function	যেকোনো Activation Function (Sigmoid, Tanh, ReLU ইত্যাদি)
Output-এর ধরন	শুধুমাত্র 0 বা 1	একটি continuous বা bounded মান
সমাধানযোগ্য সমস্যা	শুধুমাত্র Linearly Separable সমস্যা	Non-linear সমস্যাও (Activation ও একাধিক Layer-এর সাহায্যে)
ঐতিহাসিক গুরুত্ব	Neural Network-এর প্রথম মডেল	আধুনিক Neural Network-এর মৌলিক একক

গুরুত্বপূর্ণ

Perceptron থেকে Neuron-এ উত্তরণ কোনো সম্পূর্ণ নতুন ধারণা নয় — বরং এটি একই কাঠামোর একটি সম্প্রসারণ, যেখানে একটি নির্দিষ্ট Step Function-এর বদলে যেকোনো Activation Function ব্যবহারের সুযোগ তৈরি হয়। এই একটি পরিবর্তনই Neural Network-কে জটিল, Non-linear সমস্যা সমাধানের উপযোগী করে তোলে।

Chapter 2 · Activation Functions

Neural Network-এর প্রতিটি Neuron-এর ভেতরে Linear Combination গণনার পরে একটি Activation Function প্রয়োগ করা হয়। এই অধ্যায়ে আমরা দেখবো Activation Function আসলে কী, কেন এটি প্রয়োজন, এবং সবচেয়ে গুরুত্বপূর্ণ কয়েকটি Activation Function কীভাবে কাজ করে।

২.১ Activation Function

Activation Function হলো একটি গাণিতিক ফাংশন, যা Neuron-এর Linear Combination (z)-এর উপর প্রয়োগ করে একটি চূড়ান্ত Output (a) তৈরি করে।

Activation প্রয়োগের আগে ও পরে কী ঘটে তা নিচে দেখানো হলো:

$$z = w_1x_1 + \dots + w_nx_n + b$$

$$a = f(z)$$

Activation-এর আগে z একটি সাধারণ Linear মান — এর মধ্যে কোনো জটিলতা নেই। Activation Function প্রয়োগের পরে a একটি Non-linear রূপান্তরিত মান, যা Network-কে জটিল প্যাটার্ন শেখার ক্ষমতা প্রদান করে। Activation Function সাধারণত প্রতিটি Hidden Layer এবং Output Layer-এর প্রতিটি Neuron-এ ব্যবহৃত হয়।

২.২ Step Function

Step Function হলো সবচেয়ে সরল Activation Function, যা মূল Perceptron-এ ব্যবহৃত হতো। এটি একটি Threshold-এর ভিত্তিতে কাজ করে — z ধনাত্মক হলে Output 1, অন্যথায় Output 0।

$$f(z) = 1, \quad \text{যদি } z \geq 0$$

$$f(z) = 0, \quad \text{যদি } z < 0$$

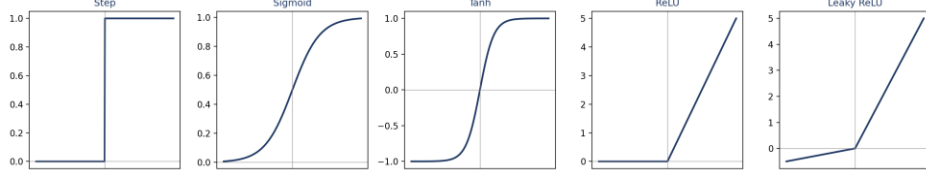
Step Function-এর সমস্যা হলো এটি একটি হঠাৎ লাফ (discontinuity) তৈরি করে এবং এর কোনো Gradient নেই ($z = 0$ ছাড়া সর্বত্র derivative শূন্য), যার ফলে এটি আধুনিক Gradient-ভিত্তিক Training-এর জন্য উপযোগী নয়।

২.৩ Sigmoid

Sigmoid Function যেকোনো বাস্তব সংখ্যাকে $(0, 1)$ সীমার মধ্যে রূপান্তরিত করে।

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid-এর Output সবসময় 0 থেকে 1-এর মধ্যে থাকে বলে একে সরাসরি Probability হিসেবে ব্যাখ্যা করা যায়। এই কারণে Binary Classification সমস্যায় Output Layer-এ Sigmoid ব্যাপকভাবে ব্যবহৃত হয় — উদাহরণস্বরূপ, একটি ছবিতে বিভাগ আছে কি না তা নির্ধারণ করার সময়।



চিত্র ২.১ — Step, Sigmoid, Tanh, ReLU এবং Leaky ReLU-এর গ্রাফ (একই স্কেলে তুলনা)।

২.৪ Tanh

Tanh (Hyperbolic Tangent) Sigmoid-এর মতোই দেখতে, তবে এর Output সীমা $(-1, 1)$ ।

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Tanh-এর একটি গুরুত্বপূর্ণ বৈশিষ্ট্য হলো এটি Zero-centered — অর্থাৎ এর Output ধনাত্মক এবং ঋণাত্মক উভয় মানই নিতে পারে, এবং $z = 0$ হলে Output-ও 0 হয়। Sigmoid-এর Output সবসময় ধনাত্মক (0 থেকে 1) হওয়ায় তা Zero-centered নয়। এই কারণে অনেক ক্ষেত্রে Tanh, Sigmoid-এর তুলনায় দ্রুত ও স্থিতিশীল Training প্রদান করে।

২.৫ ReLU

ReLU (Rectified Linear Unit) বর্তমানে Hidden Layer-এ সবচেয়ে বেশি ব্যবহৃত Activation Function।

$$f(z) = \max(0, z)$$

অর্থাৎ z ধনাত্মক হলে ReLU সেই মানটিই অপরিবর্তিত রাখে; z ঋণাত্মক হলে Output 0 হয়ে যায়। এই সরলতার কারণে ReLU গণনাগতভাবে দ্রুত, এবং এটি অনেক ক্ষেত্রে Gradient Vanishing-জনিত সমস্যা কমাতে সাহায্য করে বলে Hidden Layer-এ এটি বহুল ব্যবহৃত।

২.৬ Leaky ReLU

ReLU-এর একটি সমস্যা হলো, যখন z ক্রমাগত ঋণাত্মক থাকে, তখন Output এবং Gradient উভয়ই স্থায়ীভাবে 0 হয়ে যেতে পারে — একে বলা হয় "Dying ReLU" সমস্যা। এই অবস্থায় সংশ্লিষ্ট Neuron আর কিছু শেখে না।

Leaky ReLU এই সমস্যার সমাধানে ঋণাত্মক অঞ্চলে একটি ছোট, শূন্য নয় এমন ঢাল (slope) রাখে:

$$f(z) = z, \quad \text{যদি } z > 0$$

$$f(z) = \alpha z, \quad \text{যদি } z \leq 0 \quad (\text{সাধারণত } \alpha \approx 0.01)$$

এইভাবে z ঋণাত্মক হলেও Gradient সম্পূর্ণভাবে 0 হয় না, ফলে Neuron সম্পূর্ণভাবে "মৃত" হয়ে যায় না এবং Training চলতে থাকে।

২.৭ Softmax

Softmax একটি বিশেষ Activation Function, যা মূলত Multi-class Classification সমস্যার Output Layer-এ ব্যবহৃত হয়। এটি একাধিক Neuron-এর Raw Output (Logits)-কে একসাথে নিয়ে একটি সম্পূর্ণ Probability Distribution-এ রূপান্তরিত করে, যেখানে সমস্ত Output-এর যোগফল সবসময় 1 হয়।

এখানে Logits বলতে বোঝানো হয় Output Layer-এর Activation প্রয়োগের আগের Raw স্কোরগুলোকে।

$$P(y = i | z) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

এখানে z_i হলো i -তম Class-এর Logit, এবং হরে সকল Class-এর Logit-এর Exponential-এর যোগফল রয়েছে, যা নিশ্চিত করে যোগফল সবসময় 1 থাকে। উদাহরণস্বরূপ, তিনটি Class-এর Logits যদি হয় [2.0, 1.0, 0.1], তাহলে Softmax প্রয়োগের পর তা রূপান্তরিত হবে আনুমানিক [0.66, 0.24, 0.10]-এ — অর্থাৎ প্রথম Class-এর সম্ভাবনা সবচেয়ে বেশি।

২.৮ কেন Non-linearity প্রয়োজন?

এই অংশটি Neural Network বোঝার জন্য অত্যন্ত গুরুত্বপূর্ণ। ধরা যাক Network-এ কোনো Non-linear Activation Function ব্যবহার করা হচ্ছে না — অর্থাৎ প্রতিটি Layer শুধুমাত্র একটি Linear রূপান্তর করছে।

$$h = W_1 x + b_1$$

$$y = W_2 h + b_2$$

প্রথম সমীকরণটি দ্বিতীয় সমীকরণে বসালে পাওয়া যায়:

$$y = W_2(W_1 x + b_1) + b_2$$

এই সমীকরণটিকে সহজ করলে দেখা যায় এটি এখনো একটি সম্পূর্ণ Linear রূপান্তর:

$$y = W'x + b'$$

যেখানে $W' = W_2 W_1$ এবং $b' = W_2 b_1 + b_2$ — অর্থাৎ, যতগুলো Linear Layer-ই যোগ করা হোক না কেন, তাদের সম্মিলিত প্রভাব একটিমাত্র Linear Layer-এর সমতুল্য। ফলে একাধিক Layer যোগ করেও Network কোনো নতুন, প্রকৃত Non-linear সম্পর্ক শিখতে পারে না।

গুরুত্বপূর্ণ

Non-linear Activation Function (Sigmoid, Tanh, ReLU ইত্যাদি) প্রতিটি Layer-এর পরে প্রয়োগ করা হলে এই Collapse ঘটে না। ফলে একাধিক Layer একত্রে জটিল, বাঁকানো (curved) Decision Boundary

এবং Non-linear সম্পর্ক শিখতে সক্ষম হয়। এই কারণেই Activation Function কেবল একটি বাড়তি ধাপ নয়, বরং Neural Network-এর শেখার ক্ষমতার মূল ভিত্তি।

Chapter 3 · Neural Network Architecture

এখন পর্যন্ত আমরা একটি একক Neuron কীভাবে কাজ করে তা দেখেছি। এই অধ্যায়ে আমরা দেখবো একাধিক Neuron কীভাবে একত্রে সংগঠিত হয়ে একটি সম্পূর্ণ Neural Network তৈরি করে, এবং Input থেকে Output পর্যন্ত তথ্য কীভাবে প্রবাহিত হয়।

৩.১ Input Layer

Input Layer হলো Neural Network-এর প্রথম স্তর, যার মাধ্যমে Raw Data Network-এ প্রবেশ করে।

একটি গুরুত্বপূর্ণ বিষয় মনে রাখা প্রয়োজন: Input Layer প্রকৃত অর্থে কোনো গণনামূলক (computational) Neuron Layer নয়। এখানে কোনো Weighted Sum বা Activation গণনা করা হয় না — Input Layer কেবল Input Feature-গুলোকে ধারণ করে এবং পরবর্তী Layer-এ পাঠিয়ে দেয়।

Input Layer-এর আকার (size) সরাসরি নির্ভর করে কতগুলো Input Feature ব্যবহার করা হচ্ছে তার উপর।

৩.২ Feature

Feature হলো Input Data-এর একটি পরিমাপযোগ্য বৈশিষ্ট্য, যা Model-কে সিদ্ধান্ত নিতে সাহায্য করে। উদাহরণস্বরূপ, একটি বাড়ির দাম পূর্বাভাস দেওয়ার Model-এ Feature হতে পারে: বাড়ির আয়তন, কক্ষের সংখ্যা, অবস্থান ইত্যাদি।

Input Layer-এর আকার এবং Feature-এর সংখ্যা সবসময় সমান হয় — যদি একটি ডেটাসেটে 10টি Feature থাকে, তাহলে Input Layer-এও 10টি Node থাকবে।

৩.৩ Image Input

ছবি (Image) কীভাবে Neural Network-এর সংখ্যাসূচক Input-এ রূপান্তরিত হয়, তা বোঝা গুরুত্বপূর্ণ। একটি ছবি অসংখ্য Pixel দ্বারা গঠিত, এবং প্রতিটি Pixel-এর একটি সংখ্যাসূচক মান (Pixel Value) থাকে।

উদাহরণস্বরূপ, 20×20 আকারের একটি Grayscale ছবিতে মোট Pixel সংখ্যা:

$$20 \times 20 = 400 \text{ (Pixel values)}$$

যদি ছবিটি রঙিন (RGB) হয়, তাহলে প্রতিটি Pixel-এর জন্য তিনটি পৃথক Channel (Red, Green, Blue) থাকে, ফলে মোট মানের সংখ্যা:

$$20 \times 20 \times 3 = 1200 \text{ (Pixel values)}$$

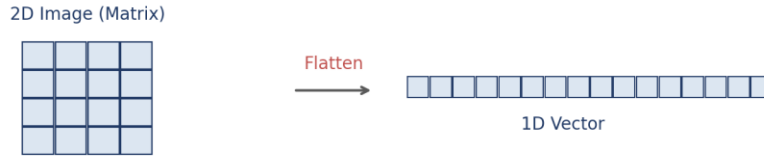
অর্থাৎ Grayscale ছবিতে প্রতি Pixel-এ একটি মান থাকে, আর RGB ছবিতে প্রতি Pixel-এ তিনটি মান থাকে — এই পার্থক্যটি Input-এর সংখ্যা নির্ধারণে গুরুত্বপূর্ণ।

৩.৪ Flatten

একটি ছবি স্বাভাবিকভাবে একটি 2D Matrix (বা RGB-এর ক্ষেত্রে 3D Matrix) আকারে থাকে। কিন্তু একটি সাধারণ Fully Connected Layer কেবল 1D Vector আকারে Input গ্রহণ করতে পারে। তাই ছবিকে Network-এ প্রবেশ করানোর আগে Flatten নামক প্রক্রিয়ায় একে একটি 1D Vector-এ রূপান্তরিত করতে হয়।

$$20 \times 20 \rightarrow \text{Flatten} \rightarrow 400$$

$$20 \times 20 \times 3 \rightarrow \text{Flatten} \rightarrow 1200$$



চিত্র ৩.১ — একটি 2D ছবিকে Flatten করে 1D Vector-এ রূপান্তর।

Flatten শুধুমাত্র আকৃতি (shape) পরিবর্তন করে — এতে কোনো মান পরিবর্তিত হয় না, শুধু সেগুলোকে একটি সরলরৈখিক ক্রমে সাজানো হয়, যাতে সেগুলো Dense Layer-এর প্রতিটি Neuron-এ পাঠানো যায়।

৩.৫ Batch Shape

Training-এর সময় Data সাধারণত একটি একটি করে নয়, বরং ছোট ছোট দলে (Batch) Network-এ প্রবেশ করানো হয়। এই কারণে Input-এর Shape সাধারণত দুইটি সংখ্যা দ্বারা প্রকাশ করা হয়:

$$(batch, features)$$

উদাহরণস্বরূপ, (32, 400) একটি সাধারণ Batch Shape, যেখানে —

32 = একটি Batch-এ কতগুলো Sample (উদাহরণ) একসাথে প্রক্রিয়াজাত করা হচ্ছে (Batch Size)।

400 = প্রতিটি Sample-এর Feature সংখ্যা (আগের 20×20 ছবির Flatten করা রূপ)।

৩.৬ Hidden Layer

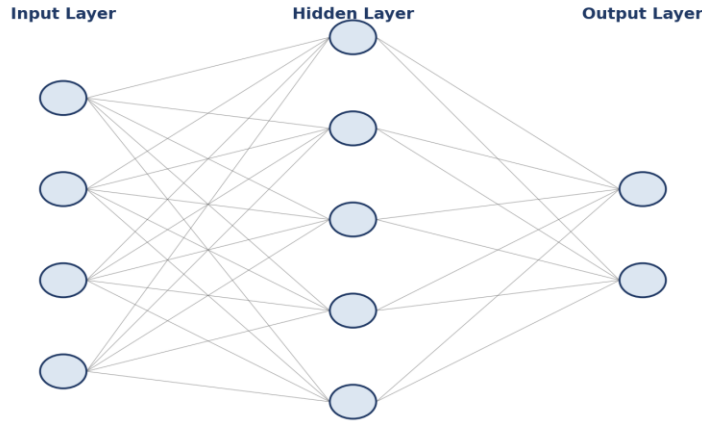
Input Layer এবং Output Layer-এর মাঝখানে অবস্থিত যেকোনো Layer-কে Hidden Layer বলা হয়। একে "Hidden" বলার কারণ হলো — এই Layer-এর মান সরাসরি বাইরে থেকে পর্যবেক্ষণযোগ্য নয়; এটি কেবল Network-এর অভ্যন্তরীণ গণনার অংশ।

প্রতিটি Hidden Layer-এ একাধিক Neuron থাকে, এবং প্রতিটি Neuron-এ পূর্ববর্তী Layer থেকে আসা মানের Weighted Sum গণনা করে তার সাথে Bias যোগ করা হয়, এবং সবশেষে একটি Activation Function প্রয়োগ করা হয়।

৩.৭ Fully Connected / Dense Layer

Fully Connected Layer (যাকে Dense Layer-ও বলা হয়) এমন একটি Layer, যেখানে পূর্ববর্তী Layer-এর প্রতিটি Node বর্তমান Layer-এর প্রতিটি Neuron-এর সাথে সংযুক্ত থাকে — এই ধরনের সংযোগকে বলা হয় All-to-All Connection।

অর্থাৎ, যদি একটি Layer-এ n -টি Input থাকে এবং পরবর্তী Layer-এ m -টি Neuron থাকে, তাহলে মোট সংযোগ সংখ্যা হবে $n \times m$ ।



চিত্র ৩.২ — একটি সম্পূর্ণ Fully Connected Neural Network: Input Layer, Hidden Layer এবং Output Layer।

৩.৮ একাধিক Neuron

একটি Layer-এর ভেতরে প্রতিটি Neuron একই Input গ্রহণ করলেও, প্রতিটির নিজস্ব আলাদা Weight এবং Bias থাকে। ফলে প্রতিটি Neuron ভিন্ন ভিন্ন গণনা করে।

$$z_1 = w_{11}x_1 + w_{12}x_2 + \dots + b_1$$

$$z_2 = w_{21}x_1 + w_{22}x_2 + \dots + b_2$$

এখানে w_{11} বোঝায় প্রথম Neuron-এর জন্য প্রথম Input-এর Weight, w_{21} বোঝায় দ্বিতীয় Neuron-এর জন্য প্রথম Input-এর Weight — এইভাবে প্রতিটি Neuron-Input জোড়ার জন্য একটি করে স্বতন্ত্র Weight থাকে।

৩.৯ ম্যাট্রিক্স আকারে প্রকাশ (Matrix Form)

একাধিক Neuron-এর এই গণনাগুলো একত্রে, আরও সংক্ষিপ্ত ও কার্যকরভাবে ম্যাট্রিক্স আকারে প্রকাশ করা যায়:

$$z = Wx + b$$

$$a = f(z)$$

এখানে W হলো একটি Weight Matrix, যার প্রতিটি সারি (row) একটি নির্দিষ্ট Neuron-এর সমস্ত Weight ধারণ করে; x হলো Input Vector; b হলো Bias Vector; এবং f হলো Activation Function, যা প্রতিটি উপাদানের উপর পৃথকভাবে (element-wise) প্রয়োগ করা হয়। এই ম্যাট্রিক্স রূপটিই বাস্তবে Neural Network বাস্তবায়নের সময় ব্যবহৃত হয়, কারণ এটি গণনাগতভাবে অনেক বেশি কার্যকর।

Chapter 4 · Parameters

Neural Network Training-এর মূল উদ্দেশ্য হলো এর Parameter-গুলোর সঠিক মান খুঁজে বের করা। এই অধ্যায়ে আমরা দেখবো Weight ও Bias আসলে কী, এবং কীভাবে একটি Network-এর মোট Parameter সংখ্যা গণনা করা হয়।

8.১ Weight

Weight হলো একটি সংখ্যা, যা নির্দেশ করে একটি নির্দিষ্ট Input পরবর্তী Neuron-এর সিদ্ধান্তে কতটা প্রভাব ফেলবে। Weight-এর মান যত বড় (Positive বা Negative দিক থেকে), সংশ্লিষ্ট Input-এর প্রভাব তত বেশি। যদি একটি Weight শূন্যের কাছাকাছি হয়, তাহলে সেই Input-এর প্রভাব প্রায় নগণ্য।

Training-এর সময় Network এই Weight-গুলোর মান ধীরে ধীরে সামঞ্জস্য করে, যাতে কোন Feature গুরুত্বপূর্ণ এবং কোনটি নয়, তা Network নিজেই শিখে নিতে পারে।

8.২ Bias

Bias হলো একটি অতিরিক্ত Parameter, যা Weighted Sum-এর সাথে যোগ করা হয়। এটি মূলত Activation Function-এর Output-কে প্রয়োজন অনুযায়ী স্থানান্তরিত (shift) করার সুযোগ দেয়।

Bias ছাড়া প্রতিটি Decision Boundary বাধ্যতামূলকভাবে মূলবিন্দু (origin) দিয়ে যেতে হতো। Bias এই সীমাবদ্ধতা দূর করে, ফলে Network আরও নমনীয়ভাবে বিভিন্ন প্যাটার্ন শিখতে পারে।

8.৩ Parameter গণনা

একটি Fully Connected Layer-এর মোট Parameter সংখ্যা নির্ভর করে সেই Layer-এর Weight এবং Bias-এর সংখ্যার উপর।

$$Parameters = Weights + Biases$$

উদাহরণস্বরূপ, যদি একটি Layer-এ 400টি Input থাকে এবং পরবর্তী Layer-এ 3টি Neuron থাকে, তাহলে

$$Weights = 400 \times 3 = 1200$$

$$Biases = 3$$

$$Total\ Parameters = 1200 + 3 = 1203$$

এই সম্পর্কটি সাধারণভাবে নিচের সূত্রে প্রকাশ করা যায়, যেখানে n হলো Input Feature-এর সংখ্যা এবং m হলো Neuron-এর সংখ্যা:

$$Parameters = n \times m + m$$

এই সূত্রটিকে আরেকটু সাজিয়ে লেখা যায়:

$$Parameters = m(n + 1)$$

একটি ছোট উদাহরণ দিয়ে যাচাই করা যাক — 2টি Input এবং 3টি Neuron থাকলে:

$$Parameters = (2 \times 3) + 3 = 6 + 3 = 9$$

গুরুত্বপূর্ণ

এই উদাহরণে সঠিক উত্তর ৯, ৮ নয়। প্রতিটি Neuron-এর নিজস্ব একটি করে পৃথক Bias থাকে — এই বিষয়টি ভুলে গেলে ভুল উত্তর (যেমন ৮) আসতে পারে। তাই Parameter গণনার সময় সবসময় Bias-এর সংখ্যা আলাদাভাবে যোগ করতে হবে।

8.8 বৃহৎ Parameter-বিশিষ্ট Model

বর্তমানে বিভিন্ন Large Language Model-কে "৫ বিলিয়ন Parameter" বা "১ ট্রিলিয়ন Parameter"-এর Model হিসেবে বর্ণনা করা হয়। এই সংখ্যাগুলো মূলত সেই Model-এর ভেতরে থাকা সমস্ত Weight ও Bias-এর মোট সংখ্যা নির্দেশ করে — এগুলো যত বেশি হয়, Model তত বেশি জটিল সম্পর্ক Represent করার সক্ষমতা রাখে।

তবে এই বইয়ের সুযোগ শুধুমাত্র Parameter-এর ধারণা এবং গণনা পদ্ধতি পর্যন্ত সীমাবদ্ধ — কোনো নির্দিষ্ট Large Model-এর স্থাপত্য (architecture) এই বইয়ের আলোচ্য বিষয় নয়।

Chapter 5 · Output Layer

Neural Network-এর Output Layer-এর গঠন সম্পূর্ণভাবে নির্ভর করে আমরা কোন ধরনের সমস্যা সমাধান করছি তার উপর। এই অধ্যায়ে চারটি প্রধান সমস্যার ধরন এবং প্রতিটির জন্য উপযুক্ত Output Layer কাঠামো আলোচনা করা হলো।

৫.১ Regression → Linear

Regression সমস্যায় Model-এর Output একটি Continuous (ধারাবাহিক) সংখ্যা — যেমন বাড়ির দাম, তাপমাত্রা ইত্যাদি পূর্বাভাস দেওয়া। এই ক্ষেত্রে Output Layer-এ একটিমাত্র Neuron থাকে, এবং তাতে কোনো Bounding Activation (যেমন Sigmoid) প্রয়োগ করা হয় না — বরং একটি Linear Activation ব্যবহার করা হয়, যাতে Output যেকোনো বাস্তব সংখ্যা হতে পারে।

৫.২ Binary Classification → Sigmoid

যখন Output মাত্র দুইটি সম্ভাব্য Class-এর একটি হতে পারে (যেমন: Cat বনাম Not Cat), তখন Output Layer-এ একটিমাত্র Neuron থাকে এবং তাতে Sigmoid Activation প্রয়োগ করা হয়। Sigmoid-এর Output (0, 1) সীমার মধ্যে থাকায়, একে সরাসরি Positive Class-এর Probability হিসেবে ব্যাখ্যা করা হয়।

৫.৩ Multi-class Classification → Softmax

যখন একটি Sample অবশ্যই K-টি Class-এর মধ্যে ঠিক একটিতে পড়বে (যেমন: হাতে লেখা সংখ্যা 0 থেকে 9 চেনা), তখন Output Layer-এ K-টি Neuron থাকে — প্রতিটি Class-এর জন্য একটি। এই K-টি Output-এর উপর Softmax Activation প্রয়োগ করা হয়, যা তাদের একটি সম্পূর্ণ Probability Distribution-এ রূপান্তরিত করে, যেখানে সব Probability-র যোগফল সবসময় 1।

৫.৪ Multi-label Classification → Sigmoid

Multi-label Classification-এ একটি Sample একই সাথে একাধিক Class-এর অন্তর্গত হতে পারে (যেমন: একটি ছবিতে একই সাথে "আকাশ", "গাছ" এবং "নদী" থাকতে পারে)। এই ক্ষেত্রে Output Layer-এ K-টি Neuron থাকে, কিন্তু প্রতিটি Neuron-এ Sigmoid স্বতন্ত্রভাবে (independently) প্রয়োগ করা হয় — অর্থাৎ একাধিক Output একসাথে উচ্চ Probability পেতে পারে, কারণ এখানে Class-গুলো একে অপরের সাথে প্রতিযোগিতা করে না।

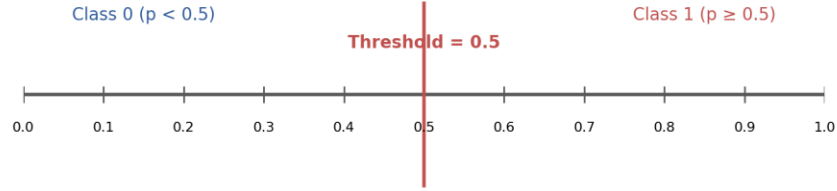
৫.৫ Probability

Activation প্রয়োগের আগের Raw Output-কে Logit বা Score বলা হয় — এটি যেকোনো বাস্তব সংখ্যা হতে পারে এবং সরাসরি ব্যাখ্যাযোগ্য নয়। Sigmoid এই Logit-কে একটি একক Probability-তে

রূপান্তরিত করে (Binary সমস্যার জন্য), আর Softmax একাধিক Logit-কে একত্রে একটি সম্পূর্ণ Probability Distribution-এ রূপান্তরিত করে (Multi-class সমস্যার জন্য)। Probability সবসময় 0 থেকে 1-এর মধ্যে থাকে এবং তা একটি নির্দিষ্ট ব্যাখ্যা বহন করে — যেমন 0.9 মানে মডেল ৯০% নিশ্চিত।

৫.৬ Threshold

Binary Classification-এ একটি Continuous Probability-কে একটি চূড়ান্ত Class-এ রূপান্তর করতে Threshold ব্যবহার করা হয়। সবচেয়ে সাধারণ Threshold হলো 0.5:



চিত্র ৫.১ — Probability থেকে Threshold-এর মাধ্যমে Class নির্ধারণ।

উদাহরণস্বরূপ —

Probability = 0.9, Threshold = 0.5 $\Rightarrow$ Class 1

Probability = 0.2, Threshold = 0.5 $\Rightarrow$ Class 0

গুরুত্বপূর্ণ

Threshold শুধুমাত্র চূড়ান্ত Prediction তৈরির সময় ব্যবহৃত হয় — অর্থাৎ Probability-কে একটি নির্দিষ্ট Class-এ রূপান্তর করার জন্য। কিন্তু Loss গণনার সময় Threshold প্রয়োগ করা Probability নয়, বরং মূল Continuous Probability মানটিই ব্যবহার করা হয়। এই পার্থক্যটি স্পষ্টভাবে বোঝা জরুরি, কারণ Threshold Training Process-এর অংশ নয়, এটি কেবল Inference বা সিদ্ধান্ত গ্রহণের সময় প্রযোজ্য।

সারাংশ সারণি

Problem	Output Neurons	Activation
Regression	1	Linear
Binary Classification	1	Sigmoid
Multi-class Classification	K	Softmax
Multi-label Classification	K	Sigmoid

Chapter 6 · Loss Functions

Neural Network কতটা ভালো পারফর্ম করছে, তা পরিমাপ করার জন্য একটি সংখ্যাসূচক মানদণ্ড প্রয়োজন। এই অধ্যায়ে আমরা Loss-এর ধারণা, এবং Regression ও Classification সমস্যার জন্য নির্দিষ্ট কিছু গুরুত্বপূর্ণ Loss Function আলোচনা করবো।

৬.১ Loss কী?

Loss হলো একটি সংখ্যাসূচক পরিমাপ, যা নির্দেশ করে Model-এর Prediction এবং প্রকৃত Target-এর মধ্যে কতটা পার্থক্য বা ত্রুটি (error) রয়েছে। Loss যত কম, Model-এর Prediction তত নির্ভুল।

৬.২ কেন Loss প্রয়োজন?

Neural Network Training-এর মূল লক্ষ্য হলো Loss-কে যতটা সম্ভব কমিয়ে আনা। Loss ছাড়া Network-এর কোনো উপায় নেই বোঝার যে তার Prediction কতটা ভুল, এবং তাই Weight ও Bias কীভাবে সামঞ্জস্য করতে হবে তাও জানা সম্ভব নয়।

সামগ্রিক প্রক্রিয়াটি নিচের ধারাবাহিকতায় ঘটে:

Prediction → Loss → Gradient → Parameter Update

এই বইয়ে আমরা কীভাবে Gradient থেকে প্রকৃতপক্ষে Parameter Update গণনা করা হয় (Backpropagation-এর বিস্তারিত প্রক্রিয়া) তা পৃথক বিষয় হিসেবে আলোচনা করছি না — বরং শুধুমাত্র ধারণাগত প্রবাহটি বোঝার উপর গুরুত্ব দেওয়া হয়েছে।

৬.৩ MSE (Mean Squared Error)

MSE সাধারণত Regression সমস্যায় ব্যবহৃত হয়।

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

এখানে y_i হলো প্রকৃত Target মান এবং $\hat{y}_i$ হলো Model-এর পূর্বাভাসিত মান। পার্থক্যকে বর্গ (square) করার কারণে বড় ত্রুটিগুলোকে ছোট ত্রুটির তুলনায় অনেক বেশি গুরুত্ব দেওয়া হয়, এবং ঋণাত্মক ও ধনাত্মক ত্রুটি একে অপরকে বাতিল করতে পারে না।

উদাহরণ: যদি প্রকৃত মান 3 এবং পূর্বাভাসিত মান 5 হয়, তাহলে ত্রুটি $(3 - 5)^2 = 4$ ।

৬.৪ MAE (Mean Absolute Error)

MAE-ও একটি Regression Loss Function, তবে এটি পার্থক্যের পরম মান (absolute value) ব্যবহার করে।

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

একই উদাহরণে, প্রকৃত মান 3 এবং পূর্বাভাসিত মান 5 হলে, ত্রুটি $|3 - 5| = 2$ । MSE বড় ত্রুটিকে বর্গ করে অতিরিক্ত গুরুত্ব দেয়, ফলে এটি Outlier-এর প্রতি বেশি সংবেদনশীল, অন্যদিকে MAE প্রতিটি ত্রুটিকে তার প্রকৃত মাত্রা অনুযায়ী সমান গুরুত্ব দেয়।

৬.৫ Huber Loss

Huber Loss মূলত MSE এবং MAE-এর মাঝামাঝি একটি সমাধান। ছোট ত্রুটির ক্ষেত্রে এটি MSE-এর মতো আচরণ করে (Squared Error), আর বড় ত্রুটির ক্ষেত্রে এটি MAE-এর মতো আচরণ করে (Absolute Error)।

এর ফলে Huber Loss ছোট ত্রুটির জন্য মসৃণ (smooth) এবং সহজে Optimize করা যায়, আবার একইসাথে বড়, ব্যতিক্রমী (outlier) ত্রুটির প্রভাবে অতিরিক্ত সংবেদনশীল হয় না — যা MSE-এর একটি সাধারণ সমস্যা।

৬.৬ Binary Cross-Entropy

Binary Classification সমস্যায় Binary Cross-Entropy সবচেয়ে বেশি ব্যবহৃত Loss Function।

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

এখানে y হলো প্রকৃত Label (0 অথবা 1), এবং p হলো Model-এর পূর্বাভাসিত Probability (Positive Class-এর জন্য)।

y -এর মান অনুযায়ী সমীকরণটি দুইটি পৃথক ক্ষেত্রে ভাগ হয়ে যায়:

$$\text{যদি } y = 1: L = -\log(p)$$

$$\text{যদি } y = 0: L = -\log(1 - p)$$

এর অর্থ বোঝা যাক উদাহরণসহ। যদি প্রকৃত Label $y = 1$ হয় এবং Model খুব আত্মবিশ্বাসের সাথে সঠিক পূর্বাভাস দেয় ($p = 0.95$), তাহলে $-\log(0.95)$ একটি খুবই ছোট মান — অর্থাৎ Loss কম। কিন্তু যদি Model ভুলভাবে খুব আত্মবিশ্বাসের সাথে ভুল পূর্বাভাস দেয় ($y = 1$ কিন্তু $p = 0.05$), তাহলে $-\log(0.05)$ একটি অনেক বড় মান — অর্থাৎ Loss অনেক বেশি।

সংক্ষেপে: সঠিক ও আত্মবিশ্বাসী Prediction ছোট Loss তৈরি করে, আর ভুল অথচ আত্মবিশ্বাসী Prediction বড় Loss তৈরি করে।

৬.৭ Cross-Entropy (Multi-class)

Multi-class Classification সমস্যায় ব্যবহৃত হয় সাধারণ Cross-Entropy Loss, যা কেবল সঠিক Class-এর জন্য পূর্বাভাসিত Probability বিবেচনা করে।

$$L = -\log(p_{\text{correct}})$$

এখানে p_{correct} হলো Model কর্তৃক প্রকৃত (সঠিক) Class-এর জন্য প্রদত্ত Probability। এই Probability যত বেশি হবে (অর্থাৎ Model যত বেশি নিশ্চিত সঠিক Class সম্পর্কে), Loss তত কম হবে। বিপরীতে, সঠিক Class-এর Probability যত কম হবে, Loss তত বেশি হবে।

সম্পূর্ণ সম্পর্কটি এভাবে সাজানো যায়:

Multi-class Classification → Softmax → Probability → Cross-Entropy Loss

গুরুত্বপূর্ণ

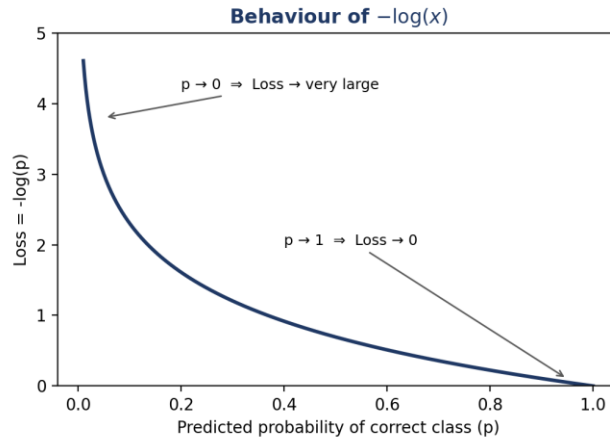
বাস্তব বাস্তবায়নে (যেমন PyTorch-এ), CrossEntropyLoss ফাংশনটি সাধারণত সরাসরি Logits নিয়ে কাজ করে, এবং প্রয়োজনীয় Softmax-সংক্রান্ত গণনা এর অভ্যন্তরেই সম্পন্ন হয়। এটি কেবল একটি বাস্তবায়নগত সুবিধা — ধারণাগতভাবে সম্পর্কটি একই থাকে: Softmax দিয়ে Probability বের করা হয়, তারপর সেই Probability থেকে Cross-Entropy Loss গণনা করা হয়।

৬.৮ $-\log(x)$ -এর আচরণ

Classification Loss-এ Negative Logarithm ব্যবহারের একটি সুনির্দিষ্ট কারণ আছে। নিচের আচরণটি লক্ষ করা যাক:

$$x \rightarrow 1 \Rightarrow -\log(x) \rightarrow 0$$

$$x \rightarrow 0 \Rightarrow -\log(x) \rightarrow \text{অত্যন্ত বড় মান}$$



চিত্র ৬.১ — $-\log(x)$ -এর গ্রাফ: সঠিক Class-এর Probability 1-এর কাছাকাছি হলে Loss প্রায় শূন্য, আর Probability 0-এর কাছাকাছি হলে Loss দ্রুত বৃদ্ধি পায়।

এই বৈশিষ্ট্যটি Classification-এর জন্য আদর্শ, কারণ এটি Model-কে কেবল সঠিক উত্তর দেওয়ার জন্য নয়, বরং সঠিক উত্তরে আত্মবিশ্বাসী হওয়ার জন্যও উৎসাহিত করে — যত বেশি নিশ্চিততার সাথে সঠিক Prediction দেওয়া হবে, Loss তত দ্রুত শূন্যের দিকে যাবে।

Chapter 7 · Gradient Descent

এটি এই বইয়ের শেষ অধ্যায়, যেখানে আমরা দেখবো কীভাবে Loss-এর মান ব্যবহার করে Neural Network প্রকৃতপক্ষে তার Weight ও Bias আপডেট করে শেখে। এই অধ্যায়টি সম্পূর্ণ বইয়ের সবগুলো ধারণাকে একত্রে সংযুক্ত করবে।

৭.১ Gradient

Gradient হলো একটি পরিমাপ, যা নির্দেশ করে একটি নির্দিষ্ট Parameter (যেমন একটি Weight) সামান্য পরিবর্তন করলে Loss কোন দিকে এবং কতটা পরিবর্তিত হবে। সহজভাবে বললে, Gradient হলো Loss পরিবর্তনের দিকনির্দেশনা।

যদি Gradient ধনাত্মক হয়, তার অর্থ Weight বাড়ালে Loss বাড়বে। আর যদি Gradient ঋণাত্মক হয়, তার অর্থ Weight বাড়ালে Loss কমবে। Gradient Descent এই তথ্য ব্যবহার করেই সঠিক দিকে অগ্রসর হয়।

৭.২ Partial Derivative

একটি নির্দিষ্ট Weight-এর সাপেক্ষে Loss-এর Gradient প্রকাশ করা হয় Partial Derivative আকারে:

$$\frac{\partial L}{\partial w}$$

এর অর্থ: "যদি Weight w সামান্য পরিবর্তিত হয়, তাহলে Loss L কীভাবে পরিবর্তিত হবে?"

এই মান ধনাত্মক হলে বোঝায় w বাড়ালে L বাড়বে (অর্থাৎ w কমানো উচিত); ঋণাত্মক হলে বোঝায় w বাড়ালে L কমবে (অর্থাৎ w বাড়ানো উচিত)। Gradient Descent সবসময় Gradient-এর বিপরীত দিকে অগ্রসর হয়, যাতে Loss ক্রমাগত হ্রাস পায়।

৭.৩ Weight Update

Gradient জানার পর, Weight-কে নিচের সূত্র অনুযায়ী আপডেট করা হয়:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

w = আপডেটের আগে এবং পরের Weight।

η (eta) = Learning Rate — একটি ছোট ধনাত্মক সংখ্যা, যা নির্ধারণ করে প্রতিবার কতটুকু পদক্ষেপে Weight পরিবর্তিত হবে।

$\partial L / \partial w$ = Gradient, যা পরিবর্তনের দিক ও মাত্রা নির্দেশ করে।

বিয়োগ চিহ্ন ($-$) নিশ্চিত করে Weight সবসময় Gradient-এর বিপরীত দিকে সরে, যাতে Loss ক্রমান্বয়ে হ্রাস পায় — Gradient-এর দিকে অগ্রসর হলে Loss বরং বৃদ্ধি পেত।

৭.৪ Bias Update

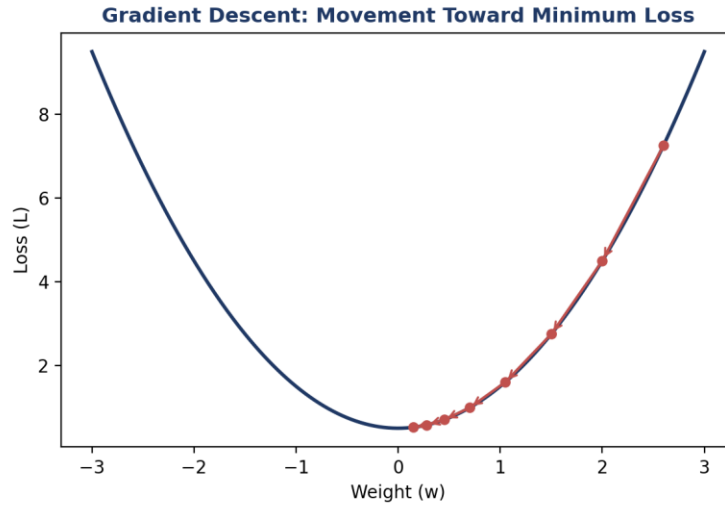
একইভাবে Bias-ও আপডেট করা হয়:

$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b}$$

এখানে ধারণাটি Weight Update-এর মতোই — শুধু পার্থক্য হলো এখানে Loss-এর Gradient Bias-এর সাপেক্ষে গণনা করা হচ্ছে।

৭.৫ Gradient Descent

Gradient Descent হলো সেই Algorithm, যা Gradient ব্যবহার করে বারবার Weight ও Bias আপডেট করতে থাকে, যতক্ষণ না Loss একটি ন্যূনতম (minimum) মানের কাছাকাছি পৌঁছায়। এর মূল লক্ষ্য হলো ধাপে ধাপে Loss-কে হ্রাস করে Model-কে ধীরে ধীরে আরও নির্ভুল করে তোলা।



চিত্র ৭.১ — Gradient Descent: প্রতিটি ধাপে Weight এমনভাবে পরিবর্তিত হয়, যাতে Loss ক্রমাগত হ্রাস পেয়ে ন্যূনতম মানের দিকে অগ্রসর হয়।

সম্পূর্ণ প্রক্রিয়ার সংযোগ

এই অধ্যায়ের শেষে সম্পূর্ণ Training প্রক্রিয়াটিকে একটি একক ধারাবাহিক প্রবাহ হিসেবে দেখা যাক:

Input → Weighted Sum → Activation → Prediction → Loss → Gradient → Gradient Descent → Weight/Bias Update

Training-এর সময় এই সম্পূর্ণ প্রক্রিয়াটি বারবার (প্রতিটি Batch বা Epoch-এ) পুনরাবৃত্তি হতে থাকে। প্রতিটি পুনরাবৃত্তিতে Weight ও Bias আরেকটু নির্ভুল হয়, এবং Loss ধীরে ধীরে হ্রাস পায় — এভাবেই একটি Neural Network ডেটা থেকে শিখে ওঠে।

Final Conceptual Summary

এই বইয়ে আমরা Perceptron-এর মতো একটি অত্যন্ত সরল ধারণা থেকে শুরু করে ধাপে ধাপে Gradient Descent পর্যন্ত একটি সম্পূর্ণ, সুশৃঙ্খল লার্নিং পথ অতিক্রম করেছি। নিচে সম্পূর্ণ প্রক্রিয়াটি একটি একক পাইপলাইন হিসেবে সংক্ষিপ্তভাবে তুলে ধরা হলো, যা দ্রুত রিভিশনের জন্য ব্যবহার করা যাবে।

Perceptron → Neuron → Activation Function → Neural Network Architecture → Parameters → Output Layer → Probability → Loss Function → Gradient → Gradient Descent → Weight/Bias Update

সম্পূর্ণ পাইপলাইনের ব্যাখ্যা

Perceptron — সবচেয়ে মৌলিক একক, যা Input, Weight, Bias নিয়ে একটি Linear Decision নেয়।

Neuron — Perceptron-এর সাধারণীকৃত রূপ, যা যেকোনো Activation Function ব্যবহার করতে পারে।

Activation Function — Non-linearity যোগ করে, যা Network-কে জটিল, বাঁকানো সম্পর্ক শেখার ক্ষমতা দেয়।

Neural Network Architecture — একাধিক Neuron ও Layer একত্রে সংগঠিত হয়ে Input থেকে Output পর্যন্ত তথ্য প্রবাহিত করে।

Parameters — প্রতিটি Weight ও Bias, যা Training-এর মাধ্যমে ক্রমাগত সামঞ্জস্য করা হয়।

Output Layer — সমস্যার ধরন (Regression, Binary, Multi-class, Multi-label) অনুযায়ী সঠিক Activation নির্বাচন করে।

Probability — Sigmoid বা Softmax-এর মাধ্যমে Logit থেকে ব্যাখ্যাযোগ্য সম্ভাবনায় রূপান্তরিত হয়।

Loss Function — Prediction ও প্রকৃত Target-এর মধ্যে পার্থক্য একটি একক সংখ্যায় পরিমাপ করে।

Gradient — Loss কীভাবে প্রতিটি Parameter-এর সাপেক্ষে পরিবর্তিত হয়, তার দিকনির্দেশ প্রদান করে।

Gradient Descent — Gradient ব্যবহার করে ধাপে ধাপে Loss হ্রাস করার Algorithm।

Weight/Bias Update — প্রতিটি ধাপে Parameter-গুলো সামান্য পরিবর্তিত হয়ে Model-কে আরও নির্ভুল করে তোলে।

চূড়ান্ত পর্যবেক্ষণ

এই বইয়ের সম্পূর্ণ কাঠামোটি একটি একক ধারণাকে কেন্দ্র করে গড়ে উঠেছে: প্রতিটি ধাপ পরবর্তী ধাপের ভিত্তি তৈরি করে। Perceptron-এর সীমাবদ্ধতা থেকে Neuron ও Activation Function-এর প্রয়োজনীয়তা তৈরি হয়েছে; একাধিক Neuron ও Layer মিলে সম্পূর্ণ Architecture গঠন করেছে; Output Layer সমস্যার ধরন অনুযায়ী Probability তৈরি করেছে; Loss সেই Probability-র ভুল পরিমাপ করেছে; এবং সবশেষে Gradient Descent সেই Loss ব্যবহার করে Model-কে ক্রমাগত উন্নত করেছে। ভবিষ্যতে এই বইটি পুনরায় পড়ার সময়, এই পাইপলাইনটি স্মরণ রাখলে সম্পূর্ণ বিষয়বস্তু দ্রুত পুনরুদ্ধার করা সম্ভব হবে।